



Capítulo 9

Ejecución Avanzada, Reporting y Preparación RFCP

Objetivos:

- Describir el propósito y la sintaxis de las opciones más importantes de la interfaz de línea de comandos de Robot Framework.
- Explicar cómo las opciones `--include`, `--exclude`, `--suite` y `--rerunfailed` controlan qué pruebas se ejecutan en cada ciclo.
- Interpretar el uso de `--variable` y las paretags para parametrizar ejecuciones y filtrar resultados de forma combinada.
- `--include` y `--exclude` filtran pruebas por etiquetas; se pueden combinar para construir selecciones precisas.

9.1: CLI Avanzado

—`--variable`, `--include`/`--exclude`, `--suite`, `--rerunfailed` y `paretags`

- Introducción

Ninguna opción de CLI modifica los archivos `.robot`. `--variable`, `--include`/`--exclude`, `--suite` y `--rerunfailed` cubren la mayoría de las necesidades de ejecución avanzada.

Categorías de Opciones del CLI de Robot Framework

Categoría	Opciones principales	Propósito
Inyección de datos	--variable, --variablefile	Pasar valores externos a las pruebas
Selección de alcance	--suite, --test	Delimitar qué suites o casos ejecutar
Filtrado por etiquetas	--include, --exclude	Seleccionar pruebas según sus tags
Reejecutar fallidos	--rerunfailed, --rerunfailedsuites	Ejecutar solo lo que falló antes

--variable: Inyección de Variables en Tiempo de Ejecución

```
# Sintaxis: --variable NOMBRE:valor
# El valor del CLI tiene precedencia sobre el definido en .robot

# Ejecutar apuntando al entorno de staging
robot --variable ENV:staging \
      --variable BASE_URL:https://staging.ejemplo.com \
      tests/

# Ejecutar contra entorno de QA
robot --variable ENV:qa \
      --variable BASE_URL:https://qa.ejemplo.com \
      tests/

# Ejecutar contra producción
robot --variable ENV:prod \
      --variable BASE_URL:https://prod.ejemplo.com \
      tests/
```

--rerunfailed: Reejecutar Solo los Casos Fallidos

```
# Primera ejecución completa
robot --output results/output.xml tests/

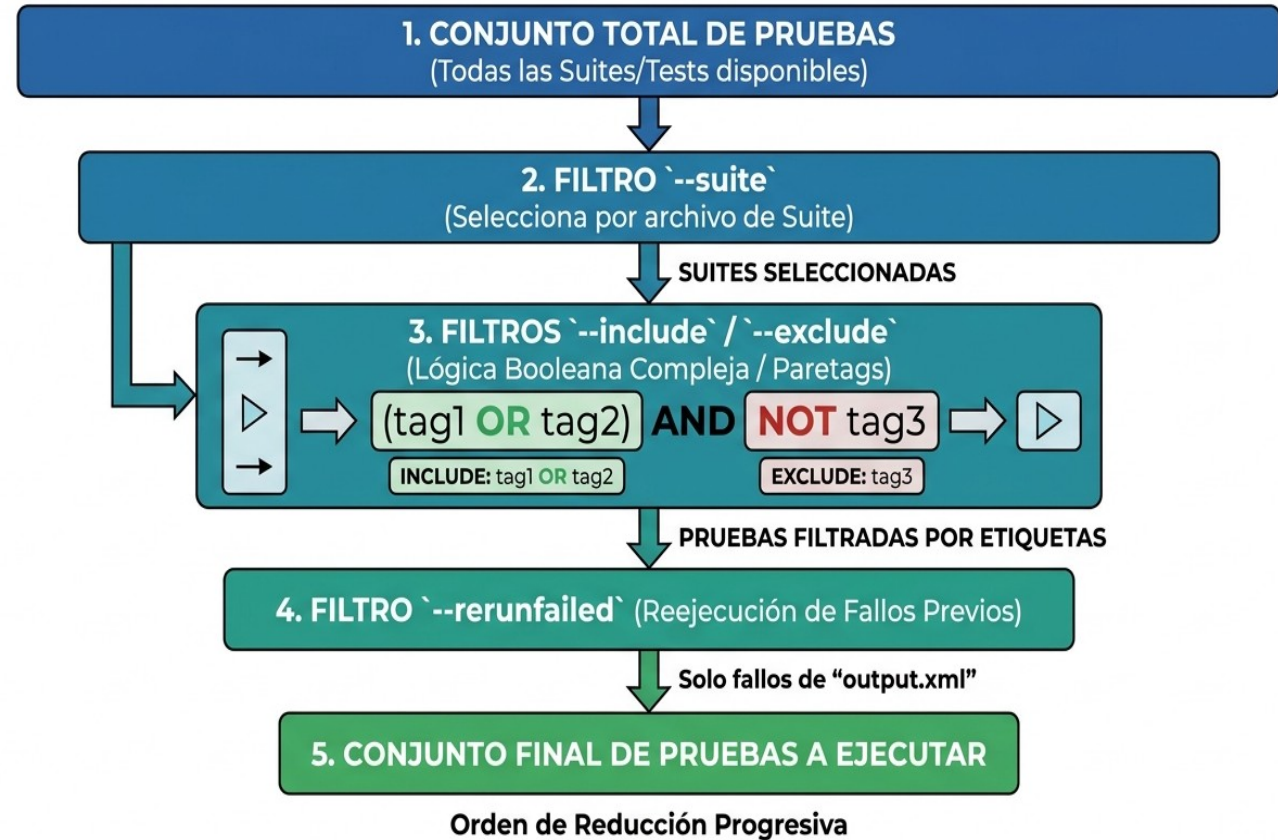
# Segunda ejecución: solo los que fallaron
robot --rerunfailed results/output.xml \
      --output results/rerun.xml \
      tests/

# Opción alternativa: reejecutar suites completas que tuvieron fallos
robot --rerunfailedsuites results/output.xml \
      --output results/rerun_suites.xml \
      tests/
```

Paretags: Combinación Lógica de Etiquetas

- ▶ Operadores: AND, OR, NOT (mayúsculas, sin espacios).
- ▶ `--include regressionANDapi` → regresión Y api.
- ▶ `--include smokeORcritical` → smoke O critical.
- ▶ `--include regressionANDNOTlento` → regresión sin lentas.

EVALUACIÓN DE FILTROS DE EJECUCIÓN EN ROBOT FRAMEWORK (ESPAÑOL)



9.2: Reporting: Análisis de report.html, log.html, métricas y

• Introducción **output.xml con rebot**

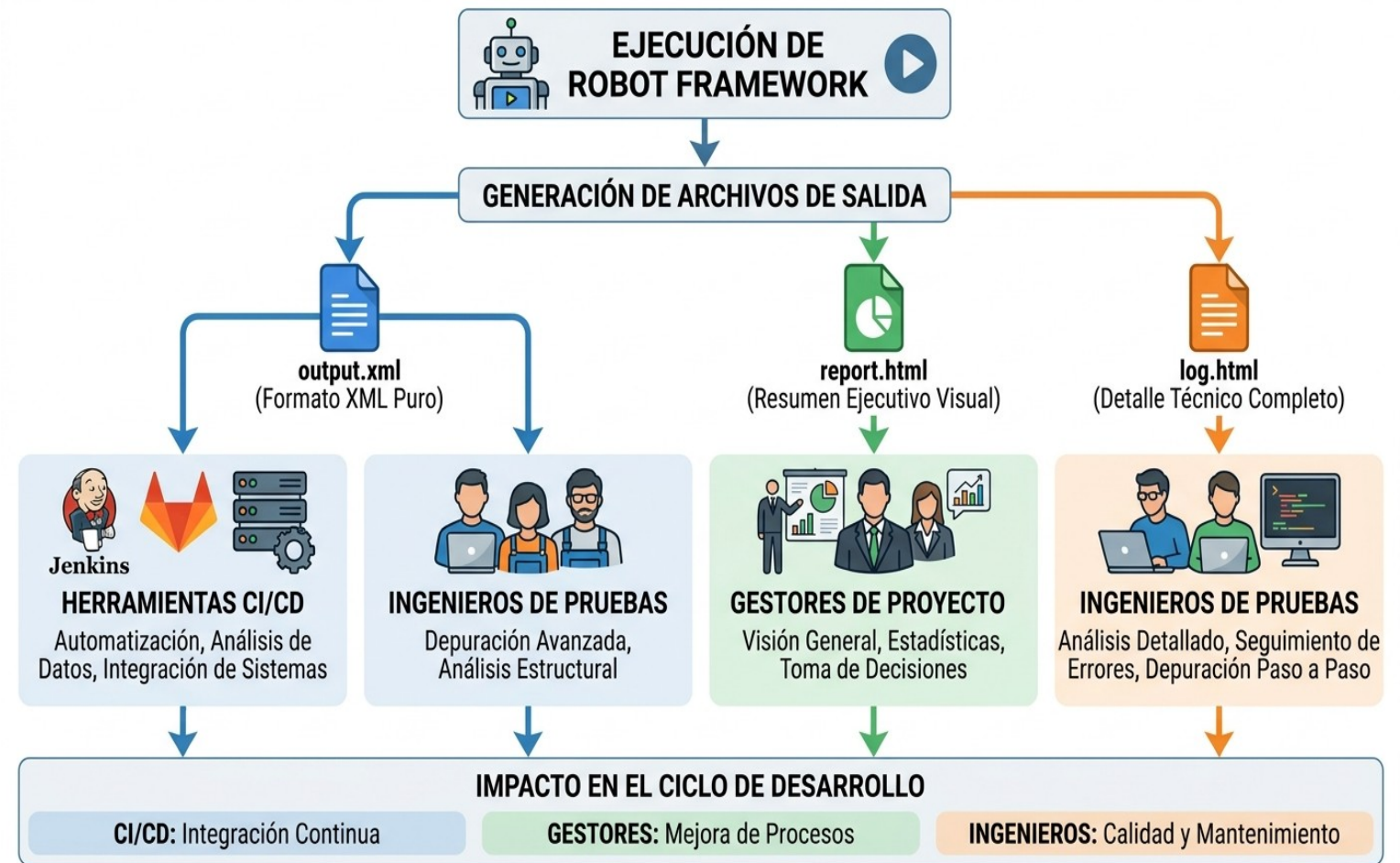
output.xml es la fuente de verdad. rebot regenera y combina reportes sin reejecutar tests — rebot --merge es la base del patrón ejecutar/reintentar/combinar.

Archivos de Salida de Robot Framework

Archivo	Propósito principal	Audiencia típica
report.html	Resumen ejecutivo de la ejecución	Gestores, QA leads
log.html	Trazabilidad detallada paso a paso	Ingenieros de automatización
output.xml	Datos estructurados de la ejecución	Herramientas, CI/CD, rebot

Anatomía de report.html y log.html

- ▶ report.html: estadísticas globales, gráfico por suite y por tag.
- ▶ report.html: lista de fallos con enlace directo a log.html.
- ▶ log.html: traza completa de cada keyword con argumentos y retornos.
- ▶ log.html: árbol expandible desde suite hasta la línea exacta de fallo.



Métricas Clave en report.html

 Inicio  Resumen  Faltos  Gráficos

INFORME DE PRUEBAS AUTOMATIZADAS



Estadísticas por Etiqueta

 PASS	PASS @smoke	Total: 4	Aprobadas: 4	Falladas: 0	Tasa de Éxito:  100% 
 FAIL	FAIL @regresion	Total: 15	Aprobadas: 12	Falladas: 3	Tasa de Éxito:  80% 
 FAIL	FAIL @api	Total: 8	Aprobadas: 5	Falladas: 3	Tasa de Éxito:  62.5% 

 Aprobado  Fallado

- ▶ Pass rate: % de casos pasados; puede bloquear despliegues en CI/CD.
- ▶ Duración por suite/caso: detecta cuellos de botella en la suite.
- ▶ Statistics by Tag: tasa de éxito por categoría funcional.
- ▶ Casos flaky: comparar múltiples output.xml con rebot.

rebot: Combinar y Filtrar Reportes

```
# Combinar dos ejecuciones paralelas en un único reporte
rebot --outputdir resultados_combinados \
      --output output_combinado.xml \
      resultados_web/output.xml \
      resultados_api/output.xml

# Paso final del pipeline: combinar resultados de 3 agentes
rebot --name "Ejecución Completa - Sprint 42" \
      --outputdir ./reportes_finales \
      agente1/output.xml \
      agente2/output.xml \
      agente3/output.xml

# Reporte filtrado: solo casos FAIL con tag "regresion"
rebot --include regresion \
      --name "Fallos en Regresión" \
      output.xml
```

9.3: Integración CI/CD

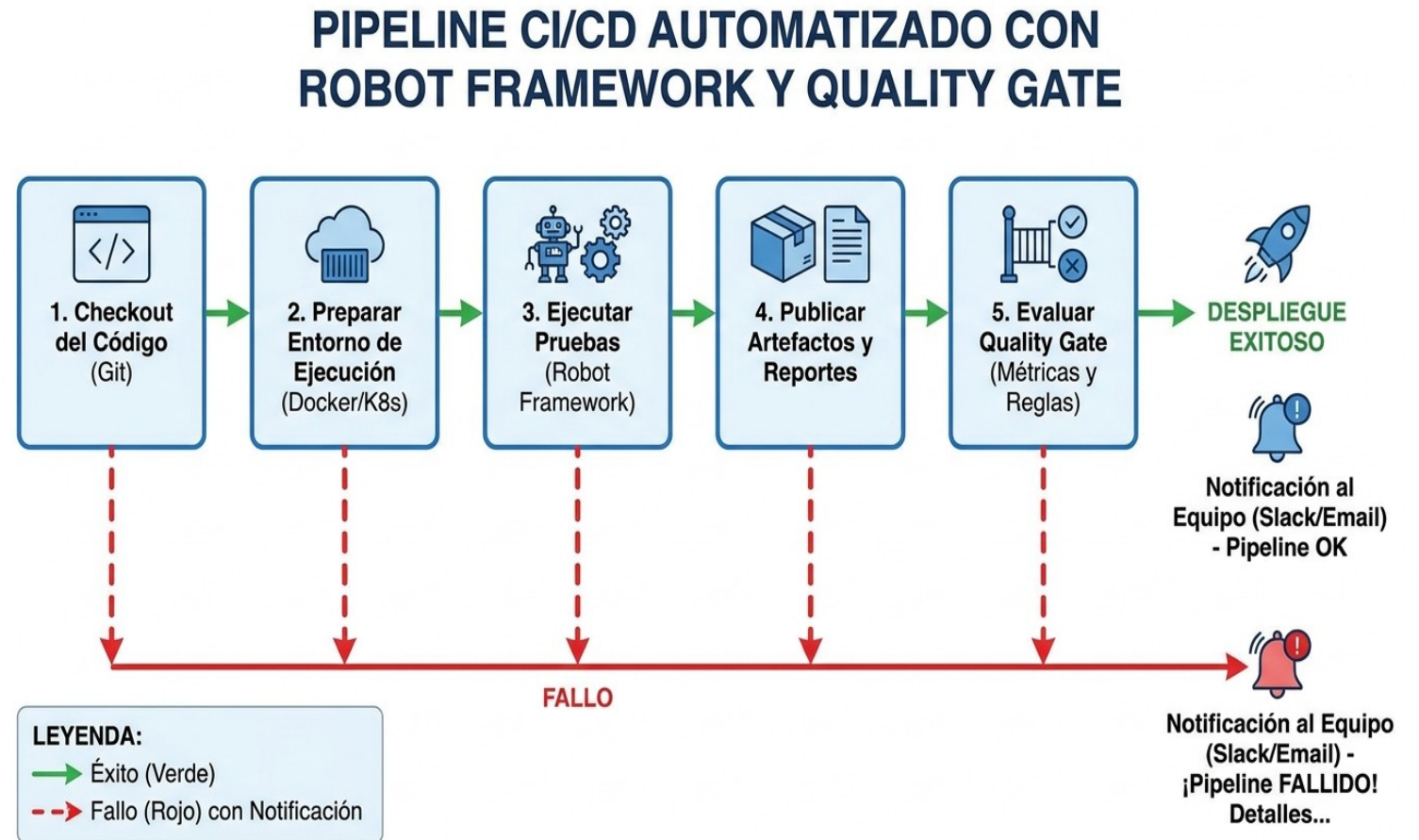
Conceptual: Pipelines GitHub Actions/Jenkins y Quality Gates

- Introducción

Un pipeline CI/CD ejecuta smoke en cada commit y regresión antes de release. Un quality gate usa el exit code de robot para decidir si el pipeline continúa.

Pipelines CI/CD y Robot Framework

- ▶ Pipeline CI/CD: secuencia automática de pasos activada por push o PR.
- ▶ robot se invoca desde CLI; solo requiere Python y las librerías instaladas.
- ▶ Exit code $\neq 0$ ante fallos: el sistema CI marca el build como fallido.
- ▶ Artefactos publicados siempre: output.xml, report.html, log.html.



GitHub Actions: Estructura del Workflow

- ▶ Plataforma CI/CD nativa de GitHub; configurada con archivos YAML en `.github/workflows/`.
- ▶ Tres niveles: Workflow (archivo YAML) → Job (runner) → Step (comando o acción).
- ▶ Eventos de activación: `on: push`, `on: pull_request`, `on: schedule`.
- ▶ Runners `ubuntu-latest` tienen Python preinstalado; no requieren infraestructura propia.
- ▶ `if: always()` en el step de publicación garantiza reportes incluso si las pruebas fallan.
- ▶ `branch protection rules` bloquean el merge del PR cuando el workflow falla.

Quality Gates: Umbrales de Calidad

Diagrama de Decisión del Quality Gate en un Pipeline de CI/CD



Legenda:

✓ = Éxito / Aprobado ✗ = Fallo / Bloqueado ◆ = Punto de Decisión ➡ = Flujo de Datos

- ▶ Quality gate: condición mínima para que el build avance o se despliegue.
- ▶ Tipos: tasa de éxito mínima, cero fallos críticos, tiempo máximo de ejecución.
- ▶ GitHub Actions: exit code $\neq 0$ ya actúa como gate básico integrado.
- ▶ Jenkins plugin: `passThreshold` (verde) y `unstableThreshold` (amarillo).

Resumen del capítulo

- `--variable NOMBRE:valor` inyecta o sobrescribe variables en tiempo de ejecución, permitiendo que la misma suite funcione en múltiples entornos sin cambiar el código.
- `--include` y `--exclude` filtran pruebas por etiquetas; se pueden combinar para construir selecciones precisas.
- `--suite` delimita la ejecución a una o varias suites por nombre, aceptando comodines con `*`.
- `--rerunfailed output.xml` ejecuta únicamente los casos que fallaron en una ejecución anterior, reduciendo drásticamente los tiempos de ciclos de corrección.
- Las **paretags** (AND, OR, NOT en mayúsculas y sin espacios) permiten expresar condiciones booleanas complejas dentro de `--include` y `--exclude`.



Resumen del capítulo

- Ninguna de estas opciones modifica los archivos .robot; toda la configuración de ejecución vive en el comando CLI o en el pipeline.
- Robot Framework genera tres artefactos al ejecutar pruebas: report.html (resumen ejecutivo), log.html (traza detallada) y output.xml (datos estructurados para máquinas).
- report.html expone métricas de tasa de éxito, duración y distribución por etiquetas, permitiendo identificar áreas funcionales con problemas.
- log.html permite navegar la traza completa de cada keyword y localizar el mensaje exacto de fallo, siendo la herramienta principal de diagnóstico para ingenieros.
- output.xml es el formato canónico de Robot Framework: puede parsearse con Python o consumirse por rebot para post-procesamiento.



Práctica 17: Ejecución avanzada por CLI con filtros de tags y regeneración de reportes

Objetivo:

Dominar la ejecución selectiva de suites por tag, el rerun automático de fallidos y la combinación de reportes con rebot.

Guía completa: [robfram-guide/Capitulo09/Practica17.md](#)



Práctica 18: Simulacro RFCP y proyecto final integrador

Objetivo:

Resolver preguntas tipo certificación RFCP y presentar un plan de adopción regional de automatización con RF.

Guía completa: [robfram-guide/Capitulo09/Practica18.md](#)



Referencias Bibliográficas

Capítulo 9: Ejecución Avanzada, Reporting y Preparación RFCP

- ▶ <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#using-command-line-options>
- ▶ <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#filtering-test-cases>
- ▶ <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#re-executing-failed-test-cases>
- ▶ <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html#setting-variables-in-command-line>
- ▶ <https://github.com/robotframework/robotframework>



¡Gracias!

¿Dudas o comentarios?

